University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: C-2

Lab task: 6

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 03/09/2026

LAB REPORT 6

Priority CPU Scheduling

(Non-Preemptive and Preemptive Implementation)

Operating System Lab | Student ID: 23101143

---

1. Objective

To implement both the non-preemptive and preemptive versions of Priority CPU scheduling in Python, and to compute and compare the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for a given set of processes, where a lower priority number indicates a higher priority.

2. Theory

2.1 Priority Scheduling — Non-Preemptive

At each scheduling point, the CPU is assigned to the process with the highest priority (lowest priority number) among all processes that have arrived and are still waiting. Once selected, a process runs to completion without interruption, even if a higher-priority process arrives in the meantime. This is simple to implement but can cause long waits for low-priority processes, and does not respond immediately to newly arrived high-priority work.

2.2 Priority Scheduling — Preemptive

The preemptive version re-evaluates the ready queue at every time unit. If a process with a higher priority (lower priority number) than the currently running process arrives, the CPU is immediately taken away from the running process and given to the new arrival; the preempted process resumes later from where it left off. This improves responsiveness for high-priority processes but increases the number of context switches and can still starve low-priority processes indefinitely.

2.3 Starvation and Aging

Both variants can starve low-priority processes if higher-priority processes keep arriving. A common remedy, aging, gradually increases the priority of a process the longer it waits, guaranteeing it will eventually run. Aging is not implemented in this lab but is worth noting as a standard extension.

3. Process Set Used

The following process set (Arrival Time and Burst Time in ms, with priority — lower number = higher priority) was used for both algorithms:

PID	AT	BT	Priority
P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Table 1: Input process set (AT = Arrival Time, BT = Burst Time, Pr = Priority)

4. Implementation Summary

Two functions were implemented:

- priority_non_preemptive(processes) — at each decision point, selects the arrived, unfinished process with the smallest priority number and runs it to completion.
- priority_preemptive(processes) — at every 1 ms tick, selects the arrived, unfinished process with the smallest priority number among those with remaining burst time, and executes it for exactly 1 ms before re-checking; a process is marked complete once its remaining burst time reaches zero.

For both, CT, TAT (= CT − AT), and WT (= TAT − BT) were computed per process, along with the average TAT and WT.

5. Execution Timeline (Gantt Chart)

5.1 Non-Preemptive

Since a running process is never interrupted, each process occupies one uninterrupted block:

Time Interval	Process Executed
0 - 3	**P1**
3 - 7	**P2**
7 - 13	**P3**
13 - 17	**P4**
17 - 19	**P5**

Table 2: CPU allocation order — non-preemptive priority scheduling

5.2 Preemptive

P1 starts first but is preempted by P2 (priority 2, higher than P1's priority 3) as soon as P2 arrives, and resumes only after P2 finishes:

Time Interval	Process Executed
0 - 1	**P1**
1 - 5	**P2**
5 - 7	**P1**
7 - 13	**P3**
13 - 17	**P4**
17 - 19	**P5**

Table 3: CPU allocation order — preemptive priority scheduling (note P1 is split into two bursts)

6. Results

6.1 Non-Preemptive Result

PID	AT	BT	Pr	CT	TAT	WT
P1	0	3	3	3	3	0
P2	1	4	2	7	6	2
P3	2	6	4	13	11	5
P4	3	4	6	17	14	10
P5	5	2	10	19	14	12
Average	-	-	-	-	**9.60**	**5.80**

Table 4: Non-preemptive priority scheduling result

6.2 Preemptive Result

PID	AT	BT	Pr	CT	TAT	WT
P1	0	3	3	7	7	4
P2	1	4	2	5	4	0
P3	2	6	4	13	11	5
P4	3	4	6	17	14	10
P5	5	2	10	19	14	12
Average	-	-	-	-	**10.00**	**6.20**

Table 5: Preemptive priority scheduling result

6.3 Comparison Summary

Metric	Non-Preemptive	Preemptive
Average TAT	9.60	10.00
Average WT	5.80	6.20

Table 6: Average TAT and WT — non-preemptive vs preemptive

7. Program Output

Non-preemptive console output:

```
Process AT   BT   Pr   CT   TAT  WT
P1      0    3    3    3    3    0
P2      1    4    2    7    6    2
P3      2    6    4    13   11   5
P4      3    4    6    17   14   10
P5      5    2    10   19   14   12

Average TAT = 9.60
Average WT = 5.80
```

Figure 1: Console output — non-preemptive priority scheduling result

Preemptive console output:

```
Process AT   BT   Pr   CT   TAT  WT
P1      0    3    3    7    7    4
P2      1    4    2    5    4    0
P3      2    6    4    13   11   5
P4      3    4    6    17   14   10
P5      5    2    10   19   14   12

Average TAT = 10.00
Average WT = 6.20
```

Figure 2: Console output — preemptive priority scheduling result

8. Discussion

P1 (priority 3) illustrates the core difference between the two variants. Under non-preemptive scheduling it runs immediately and completes at t = 3 with 0 ms waiting time, because once selected it cannot be interrupted. Under preemptive scheduling, P1 starts at t = 0 but is interrupted at t = 1 when P2 (priority 2, higher priority) arrives; P1 does not resume until P2 finishes at t = 5, so P1's completion time is pushed back to t = 7 and its waiting time rises from 0 ms to 4 ms.

P2 benefits from preemption: its waiting time drops from 2 ms (non-preemptive) to 0 ms (preemptive), since it seizes the CPU the moment it arrives instead of waiting for P1 to finish. P3, P4, and P5 are unaffected in both cases, since no higher-priority process arrives while they are ready to preempt them, and their results are identical in both tables.

Overall, the non-preemptive result gives a lower average TAT (9.60 ms vs 10.00 ms) and lower average WT (5.80 ms vs 6.20 ms) for this specific process set — slightly better than preemptive, because the extra preemption of P1 costs more waiting time than the head start it gives P2. In general, however, preemptive priority scheduling is preferred for responsiveness-critical systems, since it guarantees that high-priority processes are served immediately rather than waiting for a lower-priority process already running to finish.

9. Conclusion

Both non-preemptive and preemptive priority scheduling were implemented and tested on the same process set. Non-preemptive scheduling produced slightly better average TAT and WT here, but only because no urgent high-priority process needed to interrupt a running one at a costly moment; preemptive scheduling instead optimizes for immediate responsiveness to high-priority arrivals, at the cost of extra context switches and potentially higher waiting time for the interrupted process. Neither variant protects low-priority processes from starvation — aging would be required for that guarantee. The choice between them therefore depends on whether the system prioritizes overall average waiting time or responsiveness to high-priority work.

Githublink:
https://github.com/Moshiur1143/Operating-System-Repository-23101143-/tree/main/Lab-06-Priority-Preemptive-and-NonPreemptive/Report

10. Appendix: Python Source Code

The complete Python implementation used for this lab (both non-preemptive and preemptive priority scheduling) is listed below:

```
# -*- coding: utf-8 -*-
"""Priority Scheduling non preemptive & preemptive Implementation

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1Ol6ya9b14Ew7tlDHpeOjJJg7rrbNp_uv
"""

def priority_non_preemptive(processes):
    n = len(processes)
    completed = 0
    current_time = 0
    is_done = [False] * n
    result = []

    while completed != n:
        idx = -1
        best_priority = float('inf')

        for i in range(n):
            pid, at, bt, pr = processes[i]
            if at <= current_time and not is_done[i]:
                if pr < best_priority:
                    best_priority = pr
                    idx = i

        if idx == -1:
            current_time += 1
            continue

        pid, at, bt, pr = processes[idx]
        current_time += bt
        ct = current_time
        tat = ct - at
        wt = tat - bt

        result.append([pid, at, bt, pr, ct, tat, wt])
        is_done[idx] = True
        completed += 1

    return result


processes = [
    ["P1", 0, 3, 3],
    ["P2", 1, 4, 2],
    ["P3", 2, 6, 4],
    ["P4", 3, 4, 6],
    ["P5", 5, 2, 10]
]

result = priority_non_preemptive(processes)

result_sorted = sorted(result, key=lambda x: x[0])

print(f"{'Process':<8}{'AT':<5}{'BT':<5}{'Pr':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")
for row in result_sorted:
    pid, at, bt, pr, ct, tat, wt = row
    print(f"{pid:<8}{at:<5}{bt:<5}{pr:<5}{ct:<5}{tat:<5}{wt:<5}")

avg_tat = sum(r[5] for r in result) / len(result)
avg_wt = sum(r[6] for r in result) / len(result)
print(f"\nAverage TAT = {avg_tat:.2f}")
print(f"Average WT = {avg_wt:.2f}")

def priority_preemptive(processes):
    n = len(processes)
```

```
    remaining_bt = [p[2] for p in processes]
    completed = 0
    current_time = 0
    is_done = [False] * n
    completion_time = [0] * n

    while completed != n:
        idx = -1
        best_priority = float('inf')

        for i in range(n):
            pid, at, bt, pr = processes[i]
            if at <= current_time and not is_done[i] and remaining_bt[i] > 0:
                if pr < best_priority:
                    best_priority = pr
                    idx = i

        if idx == -1:
            current_time += 1
            continue

        remaining_bt[idx] -= 1
        current_time += 1

        if remaining_bt[idx] == 0:
            is_done[idx] = True
            completed += 1
            completion_time[idx] = current_time

    result = []
    for i in range(n):
        pid, at, bt, pr = processes[i]
        ct = completion_time[i]
        tat = ct - at
        wt = tat - bt
        result.append([pid, at, bt, pr, ct, tat, wt])

    return result


processes = [
    ["P1", 0, 3, 3],
    ["P2", 1, 4, 2],
    ["P3", 2, 6, 4],
    ["P4", 3, 4, 6],
    ["P5", 5, 2, 10]
]

result = priority_preemptive(processes)

result_sorted = sorted(result, key=lambda x: x[0])

print(f"{'Process':<8}{'AT':<5}{'BT':<5}{'Pr':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")
for row in result_sorted:
    pid, at, bt, pr, ct, tat, wt = row
    print(f"{pid:<8}{at:<5}{bt:<5}{pr:<5}{ct:<5}{tat:<5}{wt:<5}")

avg_tat = sum(r[5] for r in result) / len(result)
avg_wt = sum(r[6] for r in result) / len(result)
print(f"\nAverage TAT = {avg_tat:.2f}")
print(f"Average WT = {avg_wt:.2f}")
```